

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: INTERNET PROGRAMMING**  
**COURSE CODE: CSA4303**

---

### COURSE OUTCOME COVERED

**CO 3:** Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

---

### QUESTIONS:

**Total Marks: 30**

#### Question 1

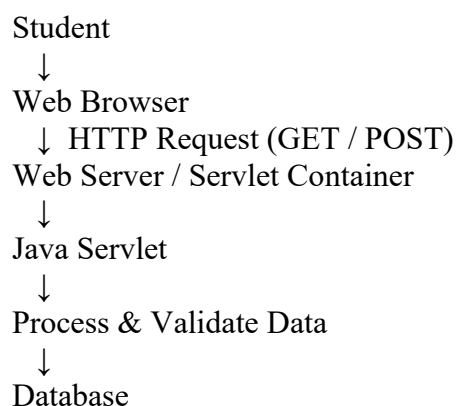
Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

##### 1. Architecture of a Servlet-Based Application

A servlet-based application follows a client–server architecture.

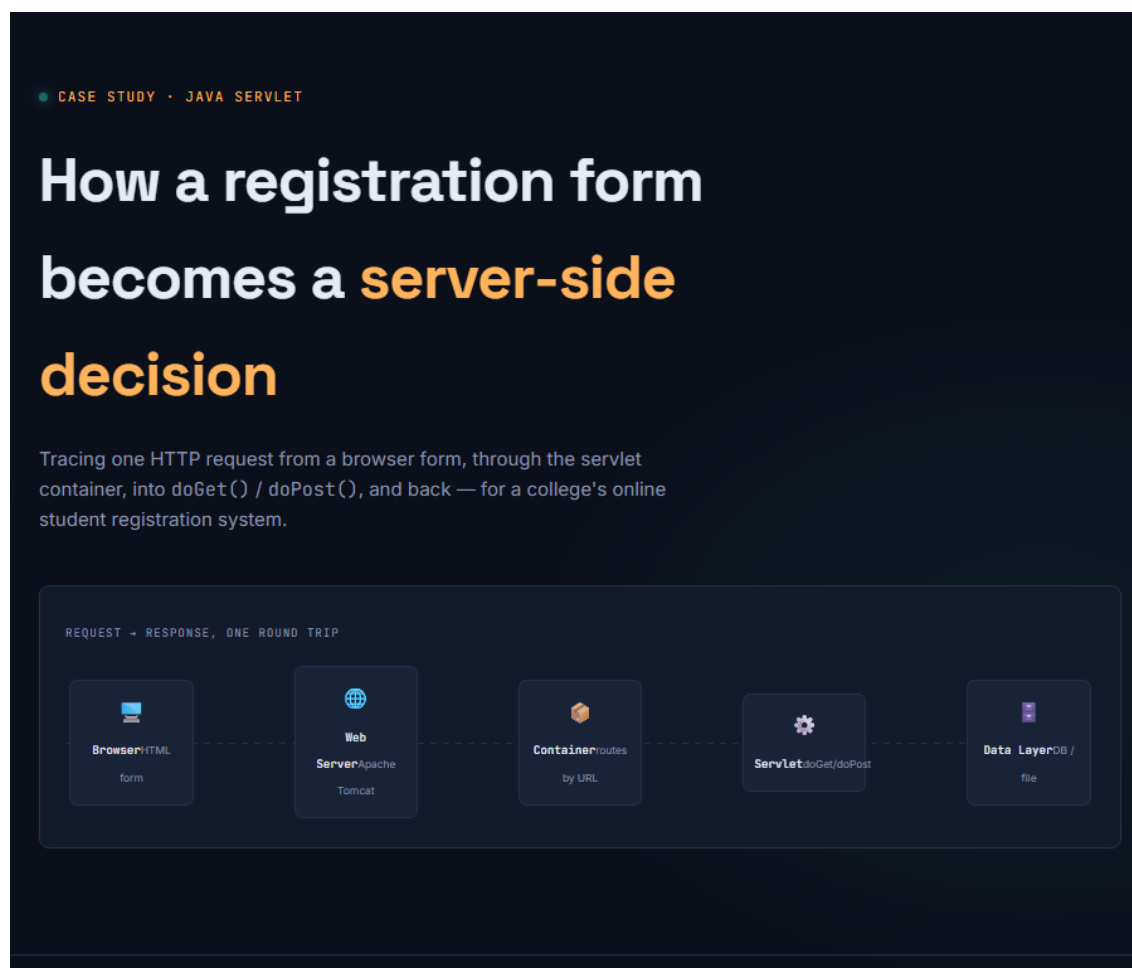
Architecture flow:



↓  
HTTP Response  
↓  
Web Browser

## Main Components

1. **Client:** The student's web browser used to enter registration details.
2. **Web Form:** Collects student information such as name, email, department, and register number.
3. **Servlet Container:** For example, Apache Tomcat. It manages servlet execution and life cycle.
4. **Servlet:** Receives the request, processes the form data, and generates a response.
5. **Database:** Stores the student's registration information.
6. **HTTP Response:** The servlet sends the result back to the browser



## init → service → destroy

A servlet instance is deliberately long-lived. The container creates it once and reuses it for thousands of requests — click a stage to see what actually happens inside it.

### STAGE 1

#### init()

runs once

Container loads the class and calls this before any request is served.

### STAGE 2

#### service()

runs per request

Dispatches to doGet, doPost, doPut, etc. based on the HTTP method.

### STAGE 3

#### destroy()

runs once

Container calls this before removing the servlet — usually at server shutdown.

**init()** — called exactly once, right after the container instantiates `RegistrationServlet`, before it can serve a single request. This is where you open the database connection pool, load config, or prepare anything expensive that should *not* happen on every request. If `init()` throws, the servlet never goes into service.

## Same servlet, two very different requests

Try both below. The right panel builds the actual HTTP request your browser would send — notice where the data goes.

### GET /register?...

- Parameters appended to the URL as a query string
- Visible in browser history, server logs, and bookmarks
- Idempotent — safe to repeat, cacheable
- Limited length (~2000 chars in practice)
- Right for: loading the empty form, filtering a course list

### POST /register

- Parameters sent in the request body, not the URL
- Not logged in server access logs or browser history by default
- Not idempotent — resubmission needs explicit confirmation
- No practical size limit for form data
- Right for: the actual registration submission

GET

POST

STUDENT NAME

Nisha K

EMAIL

nisha@saveetha.com

ROLL NUMBER

192411231

COURSE

B.Tech Computer Science



Submit registration

#### WHAT THE BROWSER ACTUALLY SENDS

GET /register?  
studentName=Nisha%20K&email=nisha%40saveetha.com&rollNumber=192411231&course=B.Tech%20Computer%20Science HTTP/1.1  
Host: college-portal.edu

▲ Visible in the address bar:  
https://college-portal.edu/register?  
studentName=Nisha%20K&email=nisha%40saveetha.com&rollNumber=192411231&course=B.Tech%20Computer%20Science

▲ Logged in server access.log verbatim.  
▲ Cached in browser history — including the email and roll number.

→ doGet() would run. Fine for loading a blank form, risky for a submission carrying personal data.

# Same servlet, two very different requests

Try both below. The right panel builds the actual HTTP request your browser would send — notice where the data goes.

## GET /register?...

- Parameters appended to the URL as a query string
- Visible in browser history, server logs, and bookmarks
- Idempotent — safe to repeat, cacheable
- Limited length (~2000 chars in practice)
- Right for: loading the empty form, filtering a course list

## POST /register

- Parameters sent in the request body, not the URL
- Not logged in server access logs or browser history by default
- Not idempotent — resubmission needs explicit confirmation
- No practical size limit for form data
- Right for: the actual registration submission

GET

POST

STUDENT NAME

Nisha K

EMAIL

nisha@saveetha.com

ROLL NUMBER

192411231

COURSE

B.Tech Computer Science

Submit registration

WHAT THE BROWSER ACTUALLY SENDS

POST /register HTTP/1.1

Host: college-portal.edu

Content-Type: application/x-www-form-urlencoded

Content-Length: 104

studentName=Nisha%20K&email=nisha%40saveetha.com&rollNumber=192411231&course=B.Tech%20Computer%20Science

✓ Address bar stays at /register — no data exposed there.

✓ Body isn't written to standard access logs.

✓ Not idempotent, so an accidental refresh won't silently duplicate the registration.

+ doPost() runs. This is the appropriate path for the actual registration submission.

## Which method belongs on sensitive data?

For a registration form carrying names, emails, and roll numbers, the life-cycle method that matters most is **service()** — specifically, making sure it routes sensitive submissions through **doPost()**, never **doGet()**.

### Why POST, not GET, for the registration submission

**URLs get logged everywhere.** A GET request's query string ends up in the browser's address bar, its history, the web server's access logs, and any proxy in between. A student's email and roll number sitting in plaintext in a shared-lab browser history is a real exposure, not a hypothetical one.

**URLs get shared by accident.** Copy-pasting or bookmarking a GET URL leaks whatever was in the query string to whoever receives that link.

**POST bodies are excluded from default logging** and don't appear in the browser history, which is precisely the confidentiality profile a registration form needs.

**Idempotence matters too.** Registration is a state-changing, one-time action. POST's non-idempotent semantics correctly signal "don't repeat this automatically," which is what prevents a refreshed page from silently registering the same student twice.

POST alone isn't encryption, though — the servlet still needs to run behind **HTTPS** so the body is encrypted in transit, and should still validate and sanitize every parameter inside **doPost()**, since POST hides data from logs but never from a malicious client.

| Feature        | GET                       | POST                      |
|----------------|---------------------------|---------------------------|
| Data location  | URL                       | Request body              |
| Visibility     | Data visible in URL       | Data not displayed in URL |
| Data size      | Limited                   | Larger data can be sent   |
| Bookmarking    | Possible                  | Not normally used         |
| Sensitive data | Not suitable              | More appropriate          |
| Typical use    | Searching/retrieving data | Form submission           |

### Conclusion

A servlet-based student registration system uses a browser → web server/servlet container → servlet → database → response architecture. `doGet()` is suitable for retrieving non-sensitive information, while `doPost()` is preferred for submitting registration data, especially when combined with HTTPS.

---

## Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using `HttpSession` and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies

A website can use session management to remember a user after login and maintain their identity while they move between multiple pages. In Java Servlets, this can be implemented using `HttpSession` and Cookies.

### 1. Session Tracking in Web Applications

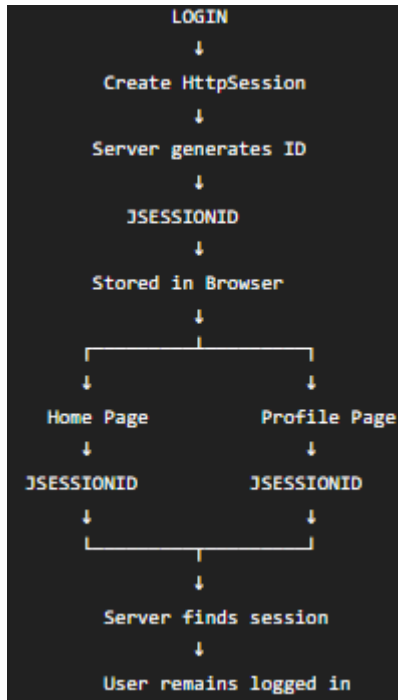
HTTP is stateless, meaning each request is treated as a new request. The server does not automatically know that two requests belong to the same user.

User Login



## 2. HttpSession and Cookies Together

In a typical servlet application, HttpSession and cookies work together.



### COMPARISON

| Feature              | HttpSession                                     | Cookies                           |
|----------------------|-------------------------------------------------|-----------------------------------|
| Storage              | Mainly server-side                              | Client/browser-side               |
| Data                 | Session attributes                              | Small pieces of data              |
| Security             | Generally safer for sensitive session data      | More exposed to client            |
| Size                 | Depends on server resources                     | Limited                           |
| Lifetime             | Until timeout/invalidation                      | Configurable expiration           |
| Server resources     | Uses server memory/storage                      | Minimal server storage            |
| Common use           | Login sessions                                  | Preferences, tracking, session ID |
| Automatic session ID | Yes                                             | Used by container for JSESSIONID  |
| User can disable     | Session tracking may fall back to URL rewriting | Yes                               |

# HTTP forgets everything.

## Sessions make it remember you.

Designing login state that survives across pages, using `HttpSession` on the server and a cookie on the client to connect one stateless request to the next.

THREE REQUESTS, ONE LOGGED-IN USER

`POST /login`

no `Cookie` header sent

Server has never seen this browser

→

`GET /dashboard`

`Cookie: JSESSIONID=8f2a91c...`

Server matches ID → finds session → knows it's Nisha

→

`GET /profile`

`Cookie: JSESSIONID=8f2a91c...`

Same ID, same session object, still Nisha

// 01 · THE PROBLEM

## Why session tracking exists at all

HTTP is stateless by design — every request is handled as if it's the first one the server has ever seen. Without something linking requests together, the server would ask the student to log in again on every single page.

### The core mechanism

On login, the server generates a unique session ID and hands it to the browser. On every later request, the browser sends that ID back, and the server uses it as a key to look up the matching state it stored earlier.

### Four ways to carry that ID

Cookies (the default), URL rewriting (`;jsessionid=...` appended to links, used when cookies are disabled), hidden form fields, and SSL session identifiers. Servlets abstract all four behind one API: `HttpSession`.

### What actually lives where

The session *ID* travels with the client (as a cookie). The session *data* — the logged-in user, cart contents, form progress — stays entirely on the server, indexed by that ID.

### Why not just use a cookie for everything?

You could store the username itself in a cookie, but then it's sitting on the client's disk, editable, and visible to anyone with browser access. Session tracking keeps the sensitive part server-side and hands the client only an opaque token.

## Watch the two stores stay in sync

Left panel: what the browser is holding. Right panel: what the server has stored, keyed by session ID. Step through login → navigate → logout.

**ACTIONS**

- 1 · Log in as Nisha K
- 2 · Visit /dashboard
- 3 · Visit /profile
- 4 · Log out

**BROWSER COOKIE JAR**

No cookies set yet.

**SERVER SESSION STORE**

No active sessions.

Click "Log in" to begin the walkthrough.

## Where the trust boundary sits

### Design rule for this login system

**Let the cookie carry only an unguessable ID.** Everything that determines what the user is allowed to do — their identity, role, permissions — stays in `HttpSession` on the server, where the client can't edit it.

**Set a real timeout.** `setMaxInactiveInterval()` bounds how long an abandoned, logged-in browser tab stays exploitable on a shared lab computer.

**Invalidate on logout, every time.** `session.invalidate()` should run even if the user just closes the tab eventually — but explicit logout is the only guaranteed trigger, which is why it's wired to its own servlet rather than left to timeout alone.

**Reserve plain cookies for non-sensitive convenience** — a remembered username, a theme preference — never for anything that gates access, since the client fully controls that value.

## 8. Advantages of HttpSession

- Easy to implement in Servlets.
- Stores user information on the server.
- Suitable for login authentication.
- Can maintain information across multiple pages.
- Session can be explicitly invalidated during logout.
- Servlet containers automatically manage session IDs.

## 9. Advantages of Cookies

- Simple client-side storage mechanism.
- Can remember information between visits.
- Useful for preferences and lightweight tracking.
- Can have a specific expiration time.

## 10. Conclusion

`HttpSession` is the preferred mechanism for maintaining authenticated user sessions in a servlet-based website. Cookies are commonly used by the servlet container to carry the `JSESSIONID` between the browser and server.

---

## Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

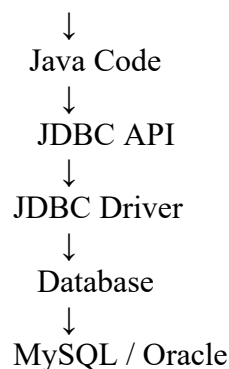
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

A Java web application can use JDBC (Java Database Connectivity) to connect to a database, insert product details, and retrieve them using SQL queries.

### 1. JDBC Architecture

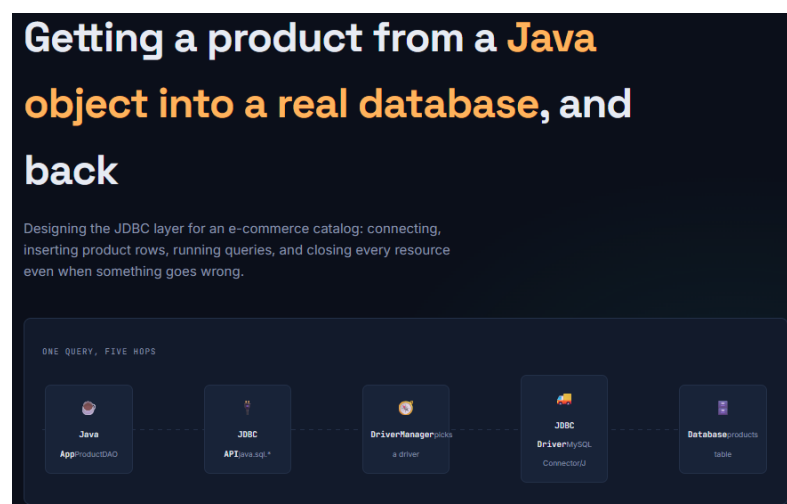
JDBC provides an interface between a Java application and a database.

#### E-Commerce Web Application



#### Main Components

1. Java Application – Sends requests to the database.
2. JDBC API – Provides classes and interfaces such as Connection, Statement, and ResultSet.
3. JDBC Driver – Converts JDBC calls into database-specific commands.
4. Database – Stores product information.



# What JDBC actually is

JDBC is not a database — it's a standard **Java API** that stays identical no matter which database sits behind it. Swap MySQL for PostgreSQL and the application code barely changes; only the driver and the connection URL do.

## JDBC API

The interfaces your code actually calls: **Connection**, **Statement**, **PreparedStatement**, **ResultSet**. Defined in **java.sql**, part of core Java.

## DriverManager

Maintains a registry of loaded drivers. Given a connection URL like **jdbc:mysql://localhost:3306/shopdb**, it picks the driver that claims to understand that URL scheme.

## JDBC Driver

A vendor-specific implementation (e.g. **mysql-connector-j**) that translates the generic JDBC calls into the database's actual wire protocol. This is the only piece that changes per database.

## The database

Receives real SQL over the driver's connection, executes it, and returns rows — which the driver then wraps back into a **ResultSet** your code can iterate.

# From zero to a result set

Every JDBC operation, insert or select, walks the same six steps.

1

## Load / register the driver

```
Class.forName("com.mysql.cj.jdbc.Driver")
```

Modern JDBC 4+ drivers self-register via **META-INF/services**, so this step is often automatic — but it's still worth knowing it's what used to be required explicitly.

2

## Establish the connection

```
DriverManager.getConnection(url, user, pass)
```

Opens a real TCP connection to the database and authenticates. This is the most expensive step — production apps use a connection pool instead of doing this per request.

3

## Create a statement

```
conn.prepareStatement(sql)
```

A **PreparedStatement** pre-compiles the SQL and lets you bind values with **?** placeholders — never string-concatenate user input into SQL.

4

## Execute

```
executeUpdate() for INSERT/UPDATE/DELETE • executeQuery() for SELECT
```

The former returns the number of rows affected; the latter returns a **ResultSet** cursor.

5

## Process the results

```
while (rs.next()) { ... }
```

Iterate the cursor and pull columns out by name or index into your Java objects (e.g. a **Product**).

6

## Close every resource

```
ResultSet → Statement → Connection
```

In reverse order of opening. This is the step most student code forgets — and exactly what try-with-resources exists to guarantee.

# What can actually go wrong

Almost every JDBC call is declared to throw `SQLException` — a checked exception that forces you to acknowledge the database can fail in ways your code can't prevent.

## Connection refused

Database is down, wrong host/port, or firewall blocking it. Caught at `DriverManager.getConnection()` — the app should fail gracefully, not crash the whole request.

## Authentication failure

Wrong username/password in the connection string. Same exception type, different `SQLState` code — worth logging separately from a network failure.

## Constraint violation

Duplicate primary key, NOT NULL violation, foreign key mismatch — thrown from `executeUpdate()`. This is expected, recoverable, and should surface a clear message to the user, not a stack trace.

## Resource leak

Not technically an exception, but the classic bug: forgetting to close a `Connection` exhausts the pool over time. Try-with-resources closes it even when the code above throws.

why try-with-resources matters

closes in reverse order, always

```
// Manual version — easy to get wrong under an exception
Connection conn = null;
try {
    conn = DriverManager.getConnection(URL, USER, PASS);
    // ... if this throws, conn.close() below is skipped entirely
} catch (SQLException e) {
    logger.error("DB error", e);
} finally {
    if (conn != null) {
        try { conn.close(); } catch (SQLException ignored) {}
    }
}

// try-with-resources — the JVM guarantees close(), even on exception
try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
    // ...
} catch (SQLException e) {
    logger.error("DB error", e);
}
```

# Insert a product, then query it back

This simulates the DAO calls above against an in-memory table — same six steps, no real database required for this walkthrough.

NEW PRODUCT

NAME

Wireless Mouse

PRICE (€)

799

STOCK

42

addProduct() — INSERT

getAllProducts() — SELECT

Step 2 `DriverManager.getConnection(URL, USER, PASS)` → connection opened

Step 3 `conn.prepareStatement("SELECT id, name, price, stock FROM products ORDER BY id")`

Step 4 `stmt.executeQuery()` → `ResultSet` returned

Step 5 `while (rs.next())` — reading 3 row(s) into `Product` objects

Step 6 `ResultSet` → `Statement` → `Connection` closed, in that order

| ID | NAME              | PRICE | STOCK |
|----|-------------------|-------|-------|
| 1  | USB-C Cable       | €249  | 120   |
| 2  | Bluetooth Speaker | €1899 | 18    |
| 3  | Wireless Mouse    | €799  | 42    |

# Designing this for production

## Beyond the basic DAO

**Use a connection pool** (HikariCP, or the app server's built-in `DataSource` ) instead of calling `DriverManager.getConnection()` per request — opening a raw TCP connection for every product lookup doesn't scale past a handful of concurrent users.

**Always use PreparedStatement** over plain `Statement` for anything built from user input — it parameterizes values instead of concatenating strings, which is what closes off SQL injection.

**Let SQLException carry a SQLState**, and branch on it — a duplicate-key error and a lost connection both throw the same class, but they call for different responses to the user.

**Never swallow an exception silently.** An empty catch block turns a database outage into a confusing "why is my product missing" bug report three steps removed from the actual cause.

## Advantages of JDBC

- Provides standard database connectivity for Java.
- Supports SQL queries.
- Works with different relational databases through appropriate drivers.
- Supports PreparedStatement.
- Provides transaction management.
- Supports exception handling through SQLException.
- Can be integrated easily with Java Servlets.

## Conclusion

JDBC acts as a bridge between a Java web application and a relational database. The basic process is load driver → establish connection → create statement → execute SQL → process results → close resources → handle exceptions. For an e-commerce application, PreparedStatement should be used for inserting and retrieving product information because it provides parameterized SQL and improves security.

## Code of Conduct:

*I Nisha K (192411231) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

## Signature of the student:

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>              | <b>Weightage (%)</b> | <b>Level 4 (Exemplary)</b>                                                | <b>Level 3 (Proficient)</b>                        | <b>Level 2 (Developing)</b>                       | <b>Level 1 (Beginning)</b>                        |
|------------------------------|----------------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%                  | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%                  | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%                  | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%                  | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%                  | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |